1. Introduction

The purpose of this case study was to build machine-learning classification models that can predict the species of an Iris flower from four measurements. The three possible species are setosa, versicolor and virginica. Classification is a type of supervised machine learning because the dataset already contains the correct species for each flower. The model learns from examples where the answer is known and then predicts the species for flowers it has not seen before.

The case study followed these stages: loading and inspecting the data, exploring it with charts, separating the input measurements from the answer, splitting the data into training and testing sets, building three models, evaluating their predictions, using cross-validation, reviewing feature importance, and comparing the models to choose a suitable final model.

2. Loading the Iris dataset

```
multi_class = spark.table("machinelearning.default.multi_class_beanchmark").toPandas()

display(multi_class)
```

The first line reads the table named multi_class_beanchmark from the Databricks workspace. The table is initially read as a Spark DataFrame. The toPandas() part converts it into a Pandas DataFrame so that we can use familiar Pandas and scikit-learn functions. The result is stored in the variable multi_class. The display() line shows the rows and columns so that we can inspect the data.

The dataset contains 150 flower records and five columns: sepal_length, sepal_width, petal_length, petal_width and species. The first four columns contain measurements. The species column contains the correct flower category that the model must learn to predict.

3. Understanding and checking the data

Before building a model, we inspect the dataset. This helps us understand what information is available and whether anything needs to be fixed.

```
multi_class.shape

multi_class.head()

multi_class.info()

multi_class.isnull().sum()

multi_class["species"].value_counts()

multi_class.duplicated().sum()

multi_class["species"].unique()
```

What each check does:

- shape reports the number of rows and columns. In this case, the dataset has 150 rows and 5 columns.
- head() displays the first five rows so we can see how the data is arranged.
- info() shows the column names, data types and how many non-missing values each column has.
- isnull().sum() counts missing values in each column. Missing values can cause problems during model training.
- value_counts() counts how many records belong to each species. The dataset has 50 examples of each of the three species.
- duplicated().sum() counts rows that are exact duplicates.
- unique() displays the different species labels. This confirms the categories the model needs to predict.

The checks showed that the dataset had no missing values and that the three species were represented equally, with 50 examples each. Equal class counts are useful because the model is not being trained on a dataset where one species greatly outnumbers the others.

4. Exploratory Data Analysis (EDA)

Exploratory Data Analysis means looking at the data before modelling. We used charts to understand the measurements and to look for patterns between the flower species.

import matplotlib.pyplot as plt

import seaborn as sns

Species counts

- sns.countplot(data=multi_class, x="species")
- plt.title("Number of flowers in each species")
- plt.show()

Histograms of the measurements

- multi_class[["sepal_length", "sepal_width", "petal_length", "petal_width"]].hist()
- plt.tight_layout()
- plt.show()

Boxplots of measurements by species

- sns.boxplot(data=multi_class, x="species", y="petal_length")
- plt.show()

Relationship between petal measurements

- sns.scatterplot(
- data=multi_class,
- x="petal_length",,y="petal_width", hue="species"),plt.show(

Correlation between numeric measurements

correlation = multi_class[

- ["sepal_length", "sepal_width", "petal_length", "petal_width"]].corr(),sns.heatmap(correlation, annot=True)
- plt.show()

The count chart checks the number of flowers in each species. Histograms show how each measurement is spread across the dataset. Boxplots compare the measurements between species and help show differences and possible unusual values. The scatter plot shows whether petal length and petal width change together and whether the species form separate groups. The correlation matrix measures how strongly pairs of numeric measurements move together. A correlation closer to 1 means a stronger positive relationship.

The charts suggested that petal length and petal width were especially useful for separating the species. Setosa is easier to distinguish, while versicolor and virginica have more overlap.

5. Separating the features and target

X = multi_class[[

"sepal_length",

"sepal_width",

"petal_length",

"petal_width"

]]

y = multi_class["species"]

The model needs to know which columns are the information it can use and which column is the answer it must predict. X contains the four flower measurements. These are called features or input variables. y contains species, which is the target or output label. We keep the target separate so the model learns to predict it from the measurements rather than simply being given the answer.

6. Splitting the data into training and testing sets

from sklearn.model_selection import train_test_split

- X_train, X_test, y_train, y_test = train_test_split(
- X, y, test_size=0.2, random_state=42, stratify=y)

The model must be tested on examples it did not use while learning. train_test_split divides the dataset into two parts. test_size=0.2 means 20% is held back for testing and 80% is used for training. With 150 records, this gives 120 training records and 30 testing records. random_state=42 makes the split repeatable, so running the code again gives the same split. stratify=y keeps the species proportions similar in both groups.

The resulting shapes were X_train (120, 4), X_test (30, 4), y_train (120,) and y_test (30,). The 4 represents the four input measurements. The 120 and 30 represent the number of flower records.

7. Building the classification models

We trained three different classification algorithms. Using more than one model allows us to compare their results instead of assuming that one method will always be best.

7.1 Decision Tree

- from sklearn.tree import DecisionTreeClassifiermodel = DecisionTreeClassifier(random_state=42)
- model.fit(X_train, y_train)
- y_pred = model.predict(X_test)

A Decision Tree makes predictions by asking a series of questions about the measurements. For example, it may ask whether petal length is below a certain value. Depending on the answer, it follows another branch until it reaches a predicted species. fit() trains the tree using the training data. predict() uses the trained tree to predict species for the test measurements.

The Decision Tree achieved 93.33% test accuracy. Its confusion matrix was [[10, 0, 0], [0, 9, 1], [0, 1, 9]]. This means it correctly predicted all 10 setosa flowers, 9 of 10 versicolor flowers and 9 of 10 virginica flowers.

7.2 Logistic Regression

- from sklearn.preprocessing import StandardScaler
- from sklearn.linear_model import LogisticRegression
- scaler = StandardScaler()
- X_train_scaled = scaler.fit_transform(X_train)
- X_test_scaled = scaler.transform(X_test)
- logistic_model = LogisticRegression(max_iter=200)
- logistic_model.fit(X_train_scaled, y_train)
- y_pred_logistic = logistic_model.predict(X_test_scaled)

Despite its name, Logistic Regression can be used for classification. It estimates the likelihood of each possible species and selects the most likely class. We scaled the measurements first because the features use different measurement ranges. StandardScaler changes each feature to a comparable scale. fit_transform() learns the scaling values from the training data and applies them to the training data. transform() applies those same training scaling values to the test data. We must not fit the scaler on the test data, because that would allow information from the test set to influence training.

max_iter=200 allows the algorithm up to 200 iterations to find its model settings. The model is trained with fit() and makes test predictions with predict(). Logistic Regression achieved 93.33% test accuracy.

7.3 Random Forest

from sklearn.ensemble import RandomForestClassifier

- random_forest = RandomForestClassifier(
- n_estimators=100,
- random_state=42)
- random_forest.fit(X_train, y_train)
- y_pred_forest = random_forest.predict(X_test)

A Random Forest builds many Decision Trees and combines their predictions. n_estimators=100 means that the forest contains 100 trees. Combining many trees can make predictions more stable than relying on only one tree. random_state=42 makes the result repeatable. The model is trained using fit() and predicts the test species using predict(). Random Forest achieved 90% test accuracy.

8. Evaluating the models

Evaluation checks how well the models predicted the correct species. Accuracy is the percentage of all test examples predicted correctly. Precision tells us, for a species predicted by the model, how many of those predictions were correct. Recall tells us how many of the actual flowers of a species the model successfully found. The F1-score combines precision and recall into one score. A higher score is generally better.

- A confusion matrix compares the actual species with the predicted species. Correct predictions appear on the main diagonal. Values away from the diagonal show which species were confused with another.
- from sklearn.metrics import (
- accuracy_score,
- classification_report,
- confusion_matrix)
- print("Accuracy:", accuracy_score(y_test, y_pred))
- print(classification_report(y_test, y_pred))
- print(confusion_matrix(y_test, y_pred))

For Logistic Regression, use y_pred_logistic in the evaluation code. For Random Forest, use y_pred_forest. The Decision Tree and Logistic Regression each achieved 93.33% accuracy, while Random Forest achieved 90%.

9. ROC-AUC and ROC curves

The ROC curve looks at how well a model separates one class from the others at different decision thresholds. AUC is the area under that curve. A value closer to 1 indicates stronger ability to rank the correct class above the other classes. Iris has three classes, so the evaluation used a one-vs-rest (OVR) approach: each species is considered in turn against the other two species.

The reported multiclass ROC-AUC values were 0.9500 for Decision Tree, 0.9967 for Logistic Regression and 0.9867 for Random Forest. Logistic Regression had the highest ROC-AUC in this experiment.

The ROC-AUC score is not the same as accuracy. Accuracy checks the final predicted labels, while ROC-AUC evaluates how well the model separates classes across possible thresholds.

10. Comparing training and testing accuracy

We also compared training accuracy with testing accuracy. Training accuracy shows how well the model predicts examples it learned from. Testing accuracy shows how well it predicts examples held back from training. A very large difference can be a warning that a model has fitted the training data too closely and may not perform as well on new data. This is sometimes called overfitting.

Decision Tree: training accuracy 100%, test accuracy 93.33%, gap 6.67 percentage points. Logistic Regression: training accuracy 95.83%, test accuracy 93.33%, gap 2.50 percentage points. Random Forest: training accuracy 100%, test accuracy 90%, gap 10 percentage points. Logistic Regression had the smallest training-to-testing gap

11. Cross-validation

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

from sklearn.pipeline import Pipeline

cv = StratifiedKFold(n_splits=5shuffle=True   random_state=42

# Decision Tree

tree_scores = cross_val_score( model, X_train, y_train, cv=cv, scoring="accuracy)

print("Decision Tree scores:", tree_scores)

print("Decision Tree average:", tree_scores.mean())

# Logistic Regression: scale inside a pipeline

logistic_pipeline = Pipeline([("scaler", StandardScaler()),   ("model", LogisticRegression(max_iter=200)])

logistic_scores = cross_val_score(  logistic_pipeline, X_train, y_train, cv=cv, scoring="accuracy")

print("Logistic Regression scores:", logistic_scores)

print("Logistic Regression average:", logistic_scores.mean())

# Random Forest
```

- forest_scores = cross_val_score(random_forest, X_train, y_train, cv=cv, scoring="accuracy")
- print("Random Forest scores:", forest_scores)
- print("Random Forest average:", forest_scores.mean())

Cross-validation tests the model several times using different parts of the training data. With five-fold cross-validation, the training data is divided into five parts. Each round uses four parts for training and the remaining part for validation. This repeats until each part has been used for validation once. The five scores are then averaged. StratifiedKFold keeps the species proportions balanced in each fold. The pipeline is important for Logistic Regression because it fits the scaler separately within each training fold, avoiding information leakage from the validation fold.The results were: Decision Tree average 95.00%; Logistic

Regression average 95.83%; Random Forest average 95.00%. Logistic Regression had the highest average cross-validation accuracy.

12. Random Forest feature importance

- importance = random_forest.feature_importances_for feature, value in zip(X.columns, importance):
- print(feature, ":", value)

Feature importance estimates how much each input feature contributed to the Random Forest's decisions. The values from this case study were: sepal_length 0.1163 (11.63%), sepal_width 0.0150 (1.50%), petal_length 0.4315 (43.15%) and petal_width 0.4372 (43.72%). The values add up to approximately 1 (or 100%). Petal width was the most important feature, closely followed by petal length. This suggests that petal measurements were more useful to this Random Forest than sepal measurements.

Feature importance describes the model's use of information; it does not prove that a feature causes a species to have a particular measurement. It is also specific to this fitted Random Forest model

13. Model comparison and selection

Measure

Decision Tree

Logistic Regression

Random Forest

Best result

Test accuracy 93.33%,93.33%,90.00%

Tie: Tree / Logistic

ROC-AUC (OVR).0.9500.0.9967.0.9867

- Logistic Regression,5-fold CV average,95.00%,95.83%,95.00%
- Logistic Regression,Train-test gap,6.67 points,2.50 points,10.00 points,

Based on the results from this experiment, Logistic Regression is the preferred model. It tied with Decision Tree for the highest test accuracy, had the highest ROC-AUC, achieved the highest cross-validation average and had the smallest training-to-testing accuracy gap. This makes it the strongest overall choice among the three models tested here. The decision is based on this dataset and these particular results; it does not mean Logistic Regression will always be best for every classification problem.

14. Final practical test: predict a new flower

The final practical step is to give the selected Logistic Regression model measurements for a new flower and ask it to predict the species. The measurements must be in the same order as the training features: sepal length, sepal width, petal length and petal width. The same scaler fitted on the training data must be used to scale the new measurements.

```
import pandas as pd

# Example measurements for one new flowe

new_flower = pd.DataFrame([{"sepal_length": 5.1,  "sepal_width": 3.5, "petal_length": 1.4,   "petal_width": 0}])

# Apply the training scaler to the new measurements

new_flower_scaled = scaler.transform(new_flower

# Predict the flower species

new_prediction = logistic_model.predict(new_flower_scaled)

print("Predicted species:", new_prediction[0])
```

This is an example input, not a result already recorded in the case study. Run the code in the notebook and record the species printed by your model. The example measurements are close to a typical setosa flower, but the model's printed prediction should be used in the submitted results.

15. Conclusion

This case study demonstrated how supervised machine learning can be used to classify Iris flowers. The dataset was inspected and explored before the four measurements were separated from the species label. The data was divided into training and testing sets, and three classification models were built: Decision Tree, Logistic Regression and Random Forest. Their performance was compared using accuracy, precision, recall, F1-score, confusion matrices, ROC-AUC, training-versus-testing accuracy and cross-validation.

Logistic Regression was selected as the preferred model because it achieved 93.33% test accuracy, the highest ROC-AUC of 0.9967, the highest cross-validation average of 95.83%, and the smallest training-to-testing accuracy gap of 2.5 percentage points. Random Forest feature importance also showed that petal length and petal width were the most influential measurements for that model. The results are promising, but the dataset is small and the test set contains only 30 flowers. The model should therefore be tested on more unseen data before being used in a real-world setting.